

PREFAIL: Identifying Precursors to Failures in Robotic Lift-and-Place Tasks to Improve Task Execution Performance

Zeyu Shangguan, Rajas Chitale, Rutvik Patel, Satyandra K. Gupta and Daniel Seita

Abstract—Non-prehensile manipulation enables flexible material handling with part carriers, but friction-based support makes high-speed motions failure-prone, while slower operation increases cycle time. Proactive failure prediction is therefore essential for efficient and reliable performance, yet existing approaches remain limited by key constraints, including sensitivity to dynamic actions and high dependence on known policy structures. Furthermore, existing methods and datasets lack a precise characterization of the latest intervention time, leaving it unclear whether a detected failure can still be prevented through timely intervention. In this paper, we investigate lift-and-place tasks for non-prehensile material handling manipulation and propose a more effective approach to predicting precursors to failures (PREFAIL) by analyzing the relative motion of target objects with respect to the carrier. We further introduce a dataset that precisely identifies the latest intervention time for risky manipulations, enabling rigorous evaluation of whether a failure prediction is actionable. We validate our approach on both simulation and real-world datasets. Our experimental results demonstrate that PREFAIL substantially improves both the accuracy and timeliness of responses to failure precursors. Our website with supplementary material is available at: <https://zshanggu.github.io/zeyu-prefail/>

I. INTRODUCTION

Non-prehensile manipulation in industrial material handling application [1], [2], [3], such as pushing, sliding, tilting, or sweeping, unlike grasping, relies solely on contact forces to support objects, where rapid direction changes introduce large inertial forces that can overcome friction, leading to failure, object damage, or costly human intervention. In contrast, operating at uniformly low speeds is impractical due to high cycle times. Therefore, it is highly desirable to have a failure prediction system that enables high-speed operation while continuously monitoring risk to achieve fast cycle times with reduced failure risk.

Current failure prediction methods demonstrate success in controlled laboratory settings [4], [5], [6], however, we observe that predicting failures during high-speed non-prehensile robot manipulation remains challenging. First, training data for failure cases during dynamic manipulation are rare. Second, we observe that current out-of-distribution (OOD) detection methods [7] struggle to distinguish between failure and non-failure trajectories in rapid manipulation scenarios. Moreover, many existing approaches hinge on predicting future actions from a learned policy. This design tightly couples failure detection to a specific control strategy, limiting cross-scenario generalization. These limitations motivate us to explore a novel approach that is (i)

Thomas Lord Department of Computer Science, University of Southern California, USA. Correspondence to: zshanggu@usc.edu and seita@usc.edu.

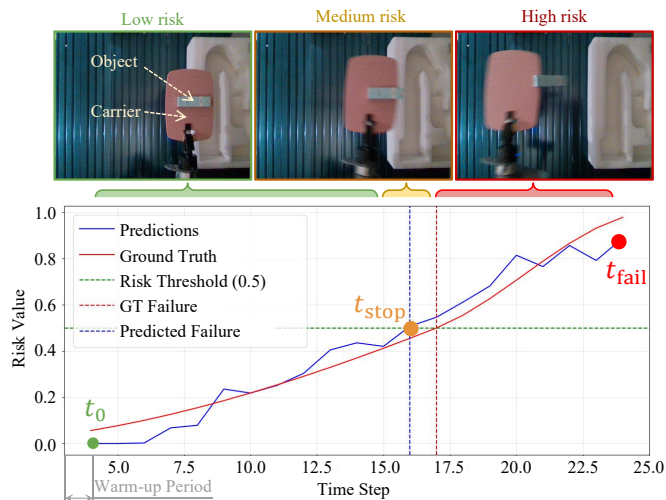


Fig. 1: Our lift-and-place task involves a carrier (the pink plate in the image) mounted on a robot arm that handles an object (the cyan box) at the maximum allowable speed. Our method utilizes the visual observation and robot state values during a real-time robotic lift-and-place task, and predicts a continuous risk value (blue solid line) that represents the likelihood of failure. Once the risk is approaching the threshold (medium risk zone), the system must adjust and slow its speed, otherwise, the failure is inevitable after t_{stop} (high risk). Since we will use sliding window during inference, there will be a warm-up period for collecting sufficient history information.

policy-agnostic and (ii) adapt to fast-moving non-prehensile manipulation scenario. In this paper, we focus on rapid lift-and-place as a representative task, which is widely used in non-prehensile material handling applications.

We hypothesize that, under fast manipulation settings, failure prediction systems must react rapidly to scene changes. Drawing an analogy from human driving: at highway speeds, drivers assess risk from surrounding vehicles' motion and adjust accordingly, but braking too late makes a collision unavoidable. This intuition suggests two key insights for robotic manipulation. First, for non-prehensile lift-and-place, unusual relative motion between objects can serve as an early indicator of impending failure. Second, there exists a critical time threshold beyond which even standard emergency intervention on industrial robots, such as emergency stop, cannot prevent failure due to inertia and contact dynamics. Existing benchmarks for failure prediction [8], [9], [10], [11] do not capture this critical temporal boundary, making it difficult to evaluate whether a system predicts failures early enough for effective intervention. To address this gap, we propose a novel data collection pipeline that precisely labels the latest moment at which an emergency stop can still prevent failure (denoted as t_{stop} , see Sec. IV). This labeled threshold enables us to evaluate whether our method predicts failures with

sufficient notice for real-time corrective action.

Motivated by these insights, we propose PREFAIL, a policy-agnostic failure prediction framework for fast lift-and-place manipulation. PREFAIL monitors geometric changes in the scene and predicts whether failures will occur before the critical deadline t_{stop} , enabling timely intervention. PREFAIL utilizes a two-branch architecture. The observation branch extracts multi-view visual features and relative object-carrier motion, and a state branch encodes proprioceptive signals and past risk estimates. These features are fused to predict a continuous risk value in $[0, 1]$. Our goal is to correctly predict the boundary risk value before t_{stop} to enable timely interventions to avoid an irrecoverable failure.

We evaluate PREFAIL using both simulation and real-world data, comparing against a competitive OOD baseline [7]. PREFAIL achieves superior performance across multiple metrics: precision, accuracy and recall for correct failure identification, and intervention timeliness ratio, a metric we introduce that measures the percentage of manipulation completed before triggering an intervention (see Sec. IV-C). A high intervention timeliness ratio indicates the system avoids overly conservative predictions that unnecessarily halts manipulation early, while still preventing failures.

Our contributions include:

- PREFAIL, a novel framework for proactive failure prediction for rapid non-prehensile lift-and-place manipulation that analyzes the relative motion between the object and the carrier.
- A dataset with precise temporal labeling that identifies the critical moment beyond which even emergency interventions cannot prevent failure, enabling evaluation of prediction timeliness in addition to accuracy.
- Experiments in simulation and the real world demonstrating that our motion-based approach achieves higher performance compared to baselines.

II. RELATED WORKS

A. Failure Detection

Current research on failure identification in robotic manipulation can be broadly categorized into two approaches: *reactive analysis* and *proactive prediction*. Prior methods for reactive analysis include failure explanation using natural language [6], [11], [12], closed set classification [13], [14], or using Large Language Models (LLMs) to aid in failure detection [15], [4], [16], [17], [18], [19]. Despite progress, their post-hoc nature limits use in industrial settings, where proactive prediction and prevention are crucial.

Compared to reactive failure detection, proactive failure prediction in robotic manipulation focuses more on real-time failure detection and prevention. Sogi et al. [20] propose a proactive success-or-failure classification framework. Enshen et al. [21] propose using geometric constraints for real-time failure prediction. Ping et al. [22] use machine learning to predict maximum object shift for wafer transferring scenarios. Parashar et al. [23] propose to combine exhaustive simulation-based falsification with Gaussian process regression to bridge the sim-to-real gap. Lee et al. [24] propose an

analytical grasp failure model for multi-suction-cup grippers that solves load distribution via a minimum spring potential energy quadratic program. Sagar et al. [25] generates action probability distributions to assess risk without explicit failure labels in fixed action space. In contrast to these scenario-specific methods, another important technique is to use out-of-distribution (OOD) methods to detect failures. Fail-Detect [7] formulate failure detection as an OOD problem to address the limitations of data collection. While prior works have made meaningful progress in failure detection, existing methods and benchmarks are evaluated on slow-speed manipulation scenarios and lack annotation of the critical intervention boundary [8], [7]; therefore, we introduce a new dataset of fast-moving manipulation.

B. OOD Methods for Failure Detection

Out-of-distribution (OOD) detection provides a general framework for robotic failure detection. Distance-based methods such as Principal Component Analysis with K-means clustering (PCA-kmeans) embed observations via PCA, flagging observations far from the in-distribution clusters as failures [26]. One-class methods like Random Network Distillation (RND) train a predictor to mimic a frozen random target network on successful trajectories, where a large mismatch at inference signals OOD behavior [27]. Consistency Flow Matching (CFM) measures trajectory curvature in the observation-to-noise forward flow, where high curvature indicates that the input is OOD [28]. Second-order methods like Natural Posterior Network (NatPN) impose a Dirichlet prior on class probabilities to separate epistemic from aleatoric uncertainty, using high epistemic uncertainty as an indicator of OOD inputs [29].

Building on these foundations, Fail-Detect [7] combines scalar OOD scores with a time-varying conformal prediction band to guarantee runtime false positive rate control, introducing the log-probability in latent noise space (logpZO) score, which maps observations to latent noise via flow matching and uses the squared norm as the OOD signal. More recently, FIPER [30] integrates an RND-based OOD score in the policy’s observation embedding space with an action-chunk entropy score over short sliding windows, triggering a failure alarm only when both exceed their conformal thresholds. As this is concurrent work, we do not include it as a baseline. However, existing OOD-based failure detection methods tend to over-confidently flag roll-outs as failure early in execution, motivating a more reliable failure detection scheme for fast-moving manipulation scenarios.

III. PROBLEM FORMULATION

We define proactive failure prediction for the lift-and-place task as an end-to-end time series prediction problem, which takes a sequence of visual observations and robot states as raw input. At each time step t_i , the system outputs a risk value $R_i \in \mathbb{R}$, representing the likelihood of a failure precursor. The observation $O_i = \{I_{t-m+1}, \dots, I_t\}_{\text{top,side}}$ contains image frames I from two camera ($C_{\text{top}}, C_{\text{side}}$) within a temporal

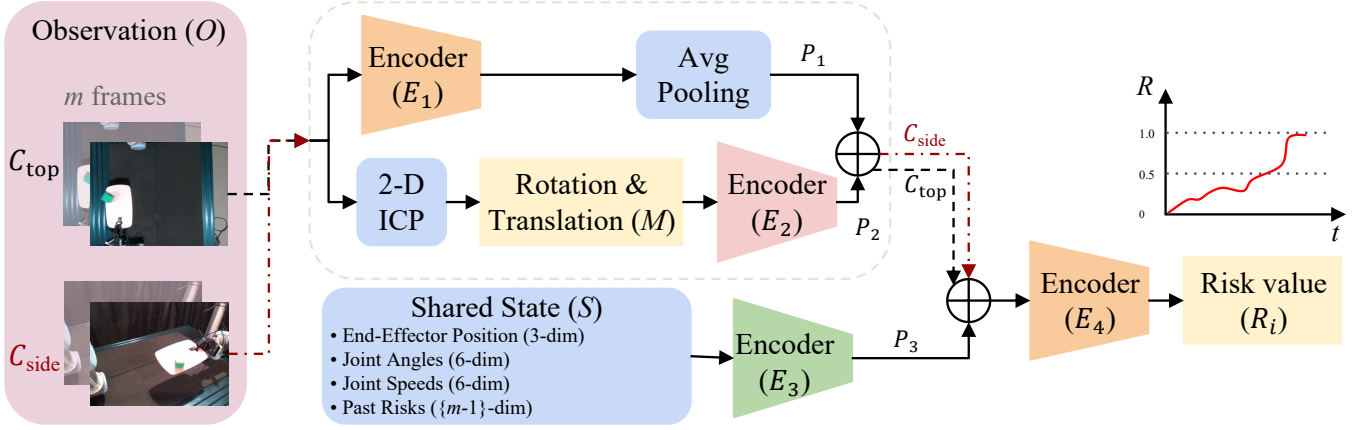


Fig. 2: **Overview of PREFAIL** (see Sec. IV for details). For each sample with m time steps, the observation branch processes multi-view images to extract visual features P_1 , then uses the segmented contours of the target object and the carrier to calculate their relative motion to get the motion features P_2 . The state branch uses E_3 to encode robot states and past risk values to produce features P_3 . We fuse and process features with E_4 to output a continuous risk value R_i indicating the likelihood of a failure at each time step t_i .

window of length m (here we use $m = 5$ empirically). The state S_i at time t_i is:

$$S_i = \{(e_{i-m+1}, \theta_{i-m+1}, \dot{\theta}_{i-m+1}), \dots, (e_i, \theta_i, \dot{\theta}_i)\}, \quad (1)$$

which contain the robot end-effector position ($e_i \in \mathbb{R}^3$), joint angles θ_i , and joint velocities $\dot{\theta}_i$ for the past m time steps. In this paper, we study *lift-and-place manipulation* in which a 6-DOF robot transports a target object o on a flat carrier c at maximum joint velocity. A failure occurs when the target object o falls from the carrier c while the robot is in motion. If the object reorients but remains on the carrier, the manipulation is considered successful. We assume at most one failure per trajectory, as a dropped object terminates the lift-and-place task.

IV. METHOD: PREFAIL

Our method, PREFAIL, consists of two input branches, as shown in Fig. 2. The observation branch extracts visual features from the multi-view video stream and the computed relative motion between the target object and the carrier. The state branch encodes robot states (from Eq. 1) along with predicted risk values from previous time steps. These features are processed to produce a continuous risk estimate.

A. Data Labeling: Historical Backtracking

We design a data collection pipeline that assigns a risk value $R_i \in [0, 1]$ to each time step during manipulation, starting from the first time step t_0 . We propose a *historical backtracking method* for data collection in simulation, which identifies two key moments within a trajectory: (1) the last time step at which an emergency stop (e-stop) can prevent a failure, denoted as t_{stop} , and (2) the moment when the failure actually occurs, denoted as t_{fail} ; for lift-and-place tasks, t_{fail} is the moment when the object drops from the carrier.

To determine t_{stop} from historical backtracking, we use counterfactual reasoning [31] and iteratively search backward from t_{fail} using a binary search strategy to find the earliest, safe intervention point. For each trajectory, we first identify t_{fail} which is detected in simulation by checking whether the

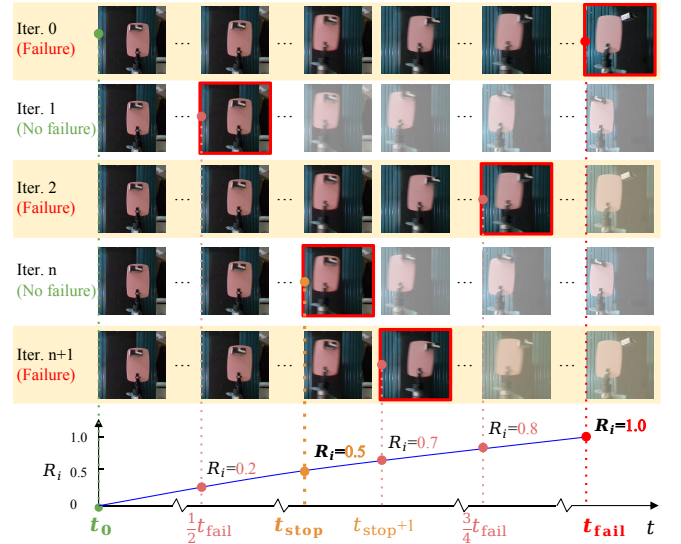


Fig. 3: **Historical backtracking data collection pipeline.** The upper portion illustrates the iterative binary search process (across all iterations: Iter. 0, 1, 2, ..., n, n+1) for historically backtracking from the actual failure moment t_{fail} (the black box drops from the pink panel), namely historical backtracking. In each iteration, we apply an e-stop (indicated by red border) and observe whether failure still occurs in the following time steps (indicated by the translucent color). The binary searching process will eventually converge to a critical time step t_{stop} , defined as the latest point at which applying an emergency stop would still prevent failure, such that the failure would not occur at t_{stop} but would inevitably occur at the subsequent timestep $t_{\text{stop}} + 1$. The curve in the bottom part represents the labeled ground truth value of the risk value, $R_i = 0.5$ at the moment t_{stop} , $R_i = 1.0$ at t_{fail} .

object's center of mass drops below that of the carrier, or manually annotated in the real world. We then iteratively narrow down t_{stop} by: (1) replaying the trajectory and applying an e-stop at the midpoint of the current search interval, (2) observing whether failure still occurs, and (3) updating the interval accordingly (searching the earlier half if failure occurs, and the later half otherwise). As shown in Fig. 3, in Iter. 1 we apply the e-stop at $\frac{1}{2}t_{\text{fail}}$ and observe no failure, so we search later; in Iter. 2 we apply it at $\frac{3}{4}t_{\text{fail}}$ and observe failure, so we search earlier. This process continues until we find t^* such that an e-stop at t^* prevents failure while one at $t^* + 1$ does not, which we define as t_{stop} . Then, for other time

points, we apply Hermite splines [32] to interpolate the risk values, enabling the model to learn the temporal progression toward the boundary (see Sec. V-G).

In the real world, historical backtracking requires physical resets of the same trajectory. Human annotators label three key moments: t_0 is the start of manipulation; t_{fail} is the moment that the object starts falling off the carrier; and t_{stop} , which is determined via a physical binary search procedure. The rollout is repeatedly replayed with the emergency stop applied at different times, and t_{stop} is the latest time at which applying the e-stop prevents failure. Because the e-stop is applied to the physical robot and its outcome directly observed, t_{stop} is empirically verified.

B. Network Design

As illustrated in Fig. 2, our network adopts a two-branch architecture. The observation branch extracts visual features from multi-view video data and the relative motion computed from consecutive images. The state branch incorporates proprioceptive robot state variables together with predicted risk values from previous time steps.

Image sequence feature. Faster relative motion often correlates with higher risk. However, such motion may be ambiguous or unobservable from certain viewpoints, such as when the object moves along the camera’s line of sight. To improve robustness, we use two cameras from distinct perspectives to capture complementary motion cues. Since robot manipulation is inherently dynamic, we conduct failure analysis over a temporal window of m steps using RGB image sequences from both cameras. The image sequence I is encoded by E_1 (here we use a CNN layer) to produce the visual features P_1 .

Relative motion feature. At each time step t_i , we obtain the relative motion values M between the target object o and the carrier c in each camera view (C_{top} and C_{side}), represented by M_{top}^i and M_{side}^i , from the image features. Specifically, $M_{\text{top,side}}^i = \{R_{o-c}^i, T_{o-c}^i\}_{\text{top,side}}$ includes an object-carrier relative rotation value ($R_{o-c}^i = \theta^i \in \mathbb{R}^1$) and an object-carrier relative translation value ($T_{o-c}^i = (X, Y)^i \in \mathbb{R}^2$). In each camera view C , the object-carrier relative rotation R^i is computed as the difference between the object-object relative rotation (R_{o-o}^i) and the carrier-carrier relative rotation (R_{c-c}^i), where the same decomposition holds for the object-carrier relative translation:

$$R_{o-c}^i = R_{o-o}^i - R_{c-c}^i, \quad T_{o-c}^i = T_{o-o}^i - T_{c-c}^i \quad (2)$$

To obtain the object-object and carrier-carrier relative motions in Eq. 2, we utilize 2D iterative closest point (ICP) [33], [34], which estimates the relative displacement between the object’s (or carrier’s) position at the current time step t_i and its position at the initial time step t_0 :

$$(R, T)_{o-o, c-c}^i = \text{ICP}(V^0, V^i) \quad (3)$$

where V^0 and V^i denote the geometric contours of the target object or carrier at the initial and current time steps, respectively. Concretely, we extract the contours $\{V_o^i, V_c^i\}_{\text{top,side}}$ in both camera views at each time step i : in real-world settings,

we leverage a pre-trained SAM2 model [35] to segment and obtain these contours, while in simulation we directly use the projected object shape as its contour. We apply 2D ICP to these contours to compute the motion values $(R, T)_{o-o, c-c}^i$, and its accuracy is further validated in the supplementary material. The relative motion makes it readily applicable to other non-prehensile manipulation tasks, since SAM2 can be used to segment arbitrary objects, instead of relying on prior knowledge of object geometry. As shown in Fig. 2, the relative motion M passes through an encoder E_2 (here we use a MLP layer) to produce the feature embeddings P_2 .

Robot state feature. In our second branch, we process the robot state S as an additional feature, which includes the end-effector position, joint angles, and joint speeds, as well as the predicted risk values from the past $m-1$ steps within the current sampling window. The first three values capture the physical state of the robot and, since they are correlated with the image observations across different camera views, this feature can be shared with the visual observation features. The past risk values encode the temporal trend of the manipulation risk, leading to more robust inference (see Sec. V-D). Although derivable from joint angles via forward kinematics, we provide the end-effector position explicitly to ease training. These states are passed through an encoder E_3 (we use a MLP layer) to produce the feature embedding P_3 .

Finally, P_1 , P_2 , and P_3 are concatenated and passed through an encoder E_4 (here we use a ResNet module [36]) to produce the final continuous risk value $R_i \in [0, 1]$. In a training batch, the samples with high risk value and the samples with low risk value are evenly mixed for balanced training. Also, the past risk is disturbed randomly for data augmentation purpose, similar to teacher forcing [37]. Details are in the supplementary website.

For a training batch with N samples, the total training loss $\mathcal{L}_{\text{Total}}$ is defined as a weighted combination of a classification loss $\mathcal{L}_{\text{Cls}}$ and a regression loss $\mathcal{L}_{\text{Reg}}$:

$$\mathcal{L}_{\text{Total}} = \alpha \mathcal{L}_{\text{Cls}} + (1 - \alpha) \mathcal{L}_{\text{Reg}}, \quad (4)$$

where α controls the relative contribution of each term. The regression loss uses mean squared error (MSE) to measure the discrepancy between the predicted risk values R_i^{pred} and their ground truth counterparts R_i^{gt} , enabling the model to learn the temporal evolution of risk,

$$\mathcal{L}_{\text{Reg}} = \frac{1}{N} \sum_{i=1}^N (R_i^{\text{pred}} - R_i^{\text{gt}})^2. \quad (5)$$

Since the learning objective is also to identify the safety boundary for triggering an emergency stop, each training sample is assigned a binary label y^i indicating whether it corresponds to a high-risk state (after t_{stop}) or a low-risk state (before t_{stop}). The classification loss is formulated as binary cross-entropy,

$$\mathcal{L}_{\text{Cls}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (6)$$

which strengthens the model’s ability to discriminate between safe and unsafe conditions, and consequently will

Methods / Variants		Simulation				Real-world			
		Precision $\uparrow$	Accuracy $\uparrow$	Recall $\uparrow$	η $\uparrow$	Precision $\uparrow$	Accuracy $\uparrow$	Recall $\uparrow$	η $\uparrow$
Fail-Detect [7] (ResNet18)	(CFM [28], FM)	0.389	0.389	1.000	0.179	0.478	0.478	1.000	0.297
	(CFM [28], DP)	0.386	0.386	1.000	0.202	0.500	0.500	1.000	0.303
	(NatPN [29], FM)	0.374	0.374	1.000	0.198	0.493	0.493	1.000	0.275
	(NatPN [29], DP)	0.384	0.384	1.000	0.185	0.493	0.493	1.000	0.275
	(PCA-kmeans [26], FM)	0.389	0.389	1.000	0.104	0.493	0.493	1.000	0.226
	(PCA-kmeans [26], DP)	0.388	0.388	1.000	0.082	0.489	0.489	1.000	0.221
	(RND [27], FM)	0.369	0.369	1.000	0.267	0.504	0.504	1.000	0.289
	(RND [27], DP)	0.384	0.384	1.000	0.170	0.507	0.507	1.000	0.302
	(logpZO [7], FM)	0.389	0.389	1.000	0.118	0.504	0.504	1.000	0.195
	(logpZO [7], DP)	0.389	0.389	1.000	0.083	0.496	0.496	1.000	0.219
PREFAIL (ours) (ResNet18)		0.991	0.995	0.995	0.933	0.944	0.963	0.985	0.932
PREFAIL (ours) (ResNet50)		0.967	0.986	0.995	0.964	0.916	0.948	0.984	0.910
PREFAIL (ours) (ResNet152)		0.995	0.997	0.995	0.923	0.900	0.933	0.978	0.925

TABLE I: Performance on simulation and real-world data. For simulation, the model is both trained and tested on simulation data. For real-world, the model is trained and tested on real-world data, with results reported as the average over 10-fold cross-validation across 10 experiments. The baseline method has multiple variants, each evaluated with different policy generation methods (FM for flow-matching [38], DP for diffusion policy [10]). The best results are highlighted in bold. Our PREFAIL consistently and largely outperforms all baseline variants. Although we do not achieve 1.000 recall, our high precision and high recall together indicate a stronger ability to distinguish failures from non-failure modes. In contrast, the baseline’s low precision with high recall suggests a biased output that defaults to predicting failure. For more details, see Sec. V-B.

Modules	Precision $\uparrow$	Accuracy $\uparrow$	Recall $\uparrow$	η $\uparrow$
Complete	0.995	0.997	0.995	0.923
Classification loss only	0.574	0.850	1.000	0.990
Regression loss only	0.980	0.991	0.995	0.855
Without images	0.973	0.988	0.995	0.904
Without relative motion	0.812	0.870	0.816	0.596
Without past risk	0.314	0.315	1.000	0.832
Without end-effector position	0.958	0.983	0.995	0.910
Without joint angles	0.927	0.972	1.000	0.860
Without joint speeds	0.384	0.417	0.995	0.839

TABLE II: Ablation results of PREFAIL trained on simulation data. We first decouple the loss terms by disabling classification or regression, then isolate the relative motion from the observation features, and then partially remove the robot state. The results indicate that the past risk values and the relative motion are the most critical components. Furthermore, both losses contribute to balancing precision, accuracy, recall, and intervention timeliness η .

improve its identification of t_{stop} and leading to a higher intervention timeliness ratio η (see Sec. IV-C and Sec. V-D).

C. Assessment Metrics

We evaluate using four metrics, precision, accuracy, recall, and intervention timeliness ratio. We calculate precision, accuracy, and recall by defining 5 cases. A true positive (TP) occurs when the model correctly predicts a failure before the latest intervention point t_{stop} , leaving sufficient time for effective action. A true negative (TN) corresponds to correctly identifying a successful rollout. A false negative (FN) occurs when the model misses a failure. False positives (FP) include two distinct cases: FP-1, where the model incorrectly reports a failure for a successful rollout, and FP-2, where the model detects a real failure only after t_{stop} , rendering the prediction too late. Examples are in Fig. 5.

In addition, to discourage overly conservative predictions, we introduce the intervention timeliness ratio η . This represents the percentage of time steps completed at the moment when the failure is correctly predicted:

$$\eta = t_{\text{stop}}^{\text{pred}} / t_{\text{stop}}, \quad (7)$$

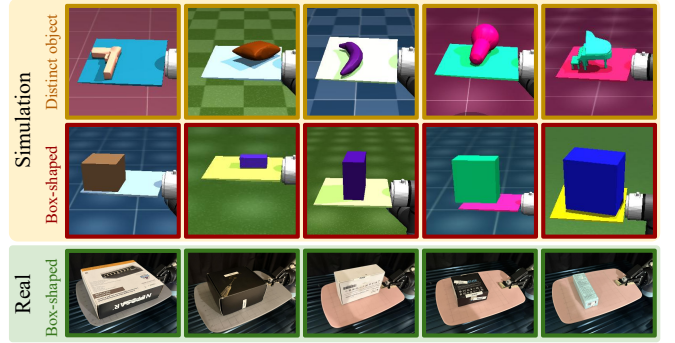


Fig. 4: Visualization of the objects and carriers used in our dataset.

where the optimal η is 1.0, indicating that the model predicts the failure at the last possible moment before the intervention deadline, thus maximizing task execution progress before triggering an emergency compensation, as shown in Fig. 5. Given the fast-moving task, a higher η is desirable, as it implies that the intervention minimizes the overall cycle time T . For real-world examples, see Sec. V-F.

V. EXPERIMENTS

A. Data and Experiment Setup

We collect two datasets to evaluate our method: one in simulation using MuJoCo [39] and one in the real world. Both use a UR5 robot arm with a flat tray carrier holding an object placed on top. The carrier is attached via a Robotiq 2F-85 parallel-jaw gripper in the real world, and rigidly fixed to the end effector through a defined joint in simulation. In the real world, two Intel RealSense D435 cameras provide top-down and side views in 30fps.

As discussed in Sec. IV-A, collecting real-world data is labor-intensive, so we collect data in simulation to facilitate large-scale verification. The **simulated** dataset has two subsets. The first focuses on box-shaped objects to mimic typical industrial handling scenarios [40] and contains 539 simulation trajectories. The second contains 5,236 simulation trajectories with diverse daily objects adopted from

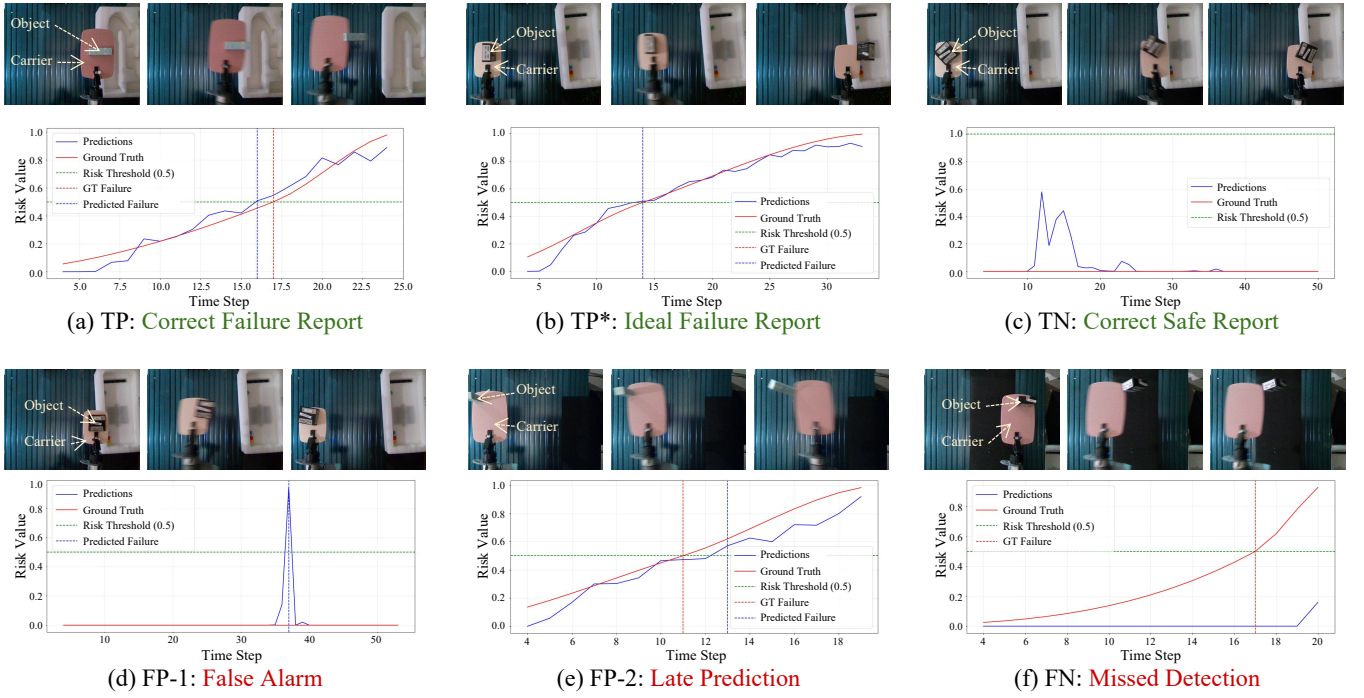


Fig. 5: Visualization of all evaluation results on real-world test cases, with the predicted risk value curve (solid red line) vs the ground truth risk value curve (solid blue line) during evaluation. The dotted red line refers to the moment of t_{stop} , while the dotted blue line is $t_{\text{stop}}^{\text{pred}}$. The threshold of risk value ($R_i = 0.5$) is indicated by dotted green line. TP* is a perfect case that demonstrates $\eta = 1.0$. See Sec. IV-C for details.

Lum et al. [41]; training and test trajectories are drawn from this set to evaluate performance across a wide range of geometries. In addition, we collect 254 **real-world** trajectories using five types of box-shaped objects. Two boxes include packaged goods with shifting contents, which introduces more variability. Experiments are conducted separately on simulation and real-world data. For each dataset, we split the collected trajectories into training, validation, and test sets with an 8:1:1 ratio (see Fig. 4 for examples). See supplementary website for detailed description of the dataset.

Each trajectory corresponds to a rapid lift-and-place task in which the robot dynamically moves the carrier (with an object on it) within the workspace, operating at its **maximum** joint velocity. To simulate challenging industrial conditions and increase trajectory diversity, we randomize object and physics parameters during data collection; details are provided in the supplementary website.

B. Model Evaluation on Simulated Data

We train and evaluate both the baseline method [7] and our model on the 5,775 simulated trajectories. All methods are trained for 200 epochs, with results reported in Tab. I. As described in Sec. IV-C, the four metrics assess performance from complementary perspectives. Compared to the OOD baseline, PREFAIL significantly improves overall performance. For example, precision improves by over 155% (from 0.389 to 0.995) and accuracy by over 156% (from 0.389 to 0.997), reducing respective error rates by more than 99%, demonstrating that PREFAIL substantially reduces false positives while correctly identifying true failure cases.

Although the baseline achieves a recall of 1.000 on our dataset, this comes with very low precision and accuracy. No-

tably, its precision and recall are identical, indicating that the baseline predicts every trajectory as failure. Such behavior is overly conservative and fails to distinguish between true positives (TP) and true negatives (TN). In contrast, PREFAIL achieves both high precision and high recall, showing that it can effectively discriminate between failure and non-failure trajectories. In addition, PREFAIL improves the execution-aware metric η from 0.198 to 0.964, indicating substantially more accurate localization of the critical stopping time t_{stop} .

C. Model Evaluation on Real-world Data

We train and evaluate our model on 254 real-world trajectories. To ensure robustness given the limited data size, we perform a ten-fold cross-validation by constructing three different splits following the same ratio and report the average performance in Tab. I.

D. Component Analysis

To evaluate the contribution of each module in our network design, we perform ablation studies using the ResNet152 backbone on the simulation dataset from three perspectives: (1) loss design, (2) observation inputs, and (3) robot state inputs. The results are summarized in Tab. II.

Loss design. We analyze the individual effects of the classification and regression losses. As shown in Tab. II, using only classification loss results in lower precision and accuracy but a high intervention timeliness ratio, whereas using only regression loss yields high precision and accuracy but a lower intervention timeliness ratio. These findings are consistent with our analysis in Sec. IV-B, where the classification objective primarily improves failure discrimi-

nation and enhances localization of t_{stop} , while the regression objective temporal evolution of the risk value.

Observation inputs. We evaluate the contributions of image sequences and relative motion features. As reported in Tab. II, using relative motion or image sequences alone does not achieve optimal performance. Combining both modalities improves performance, indicating that visual and motion cues provide complementary information, with relative motion playing a more dominant role.

Robot state inputs. We train the model by excluding one robot state at a time to analyze the contribution of each input component. Tab. II shows that past risk values and joint speed data contribute significantly to the overall performance.

E. Visualization

In Fig. 5, we visualize representative prediction results and simultaneously illustrate the five cases used to calculate precision and recall. These results are obtained under Sim&Real training and evaluation using ResNet-18. In the TN case, our model correctly predicts low risk values throughout the manipulation. In the FP case, the model either incorrectly predicts a high risk value when no failure occurs, or it predicts $t_{\text{stop}}^{\text{pred}}$ after the ground-truth t_{stop} . In the TP case, the model correctly predicts $t_{\text{stop}}^{\text{pred}}$ before the ground-truth t_{stop} . In the FN case, the model fails to predict any high risk values, even though the ground truth indicates a failure. Additionally, the TP* case represents an optimal prediction, where $t_{\text{stop}}^{\text{pred}} = t_{\text{stop}}$ and $\eta = 1.0$.

F. Real-world Deployment

To validate real-world applicability, we deployed PREFAIL on the lift-and-place task (Sec. III) and used its predicted risk to proactively prevent failures.

In a lift-and-place manipulation that potentially fails before a total cycle time of T , our goal is to intervene at an early stage by modulating the joint speed to prevent the failure, incurring an additional cycle time ΔT . For example, the original rollout failed around 0.86 s, although it was expected to finish in 1.2 s. With our speed control strategy, the robot instead completes the task successfully in 1.3 s, incurring only a 0.1 s increase in cycle time. By contrast, slowing down from the very beginning would require at least 1.8 s to avoid failure. We use the predicted risk to dynamically modulate joint velocities. At risk 0.0, the robot operates at maximum speed; as risk increases, speed decreases proportionally, reaching the minimum at risk 0.5. Speeds in between are determined by linear interpolation, as shown in Fig. 6. This risk-aware velocity scaling enables the system to respond adaptively to potential failure states, slowing down cycle in higher-risk scenarios to improve task robustness while maintaining efficiency in low-risk conditions.

G. Discussion

Limitations. PREFAIL has several limitations that present opportunities for future work. First, we study a single object on a carrier. Extending PREFAIL to multi-object scenarios would require reasoning about inter-object interactions and

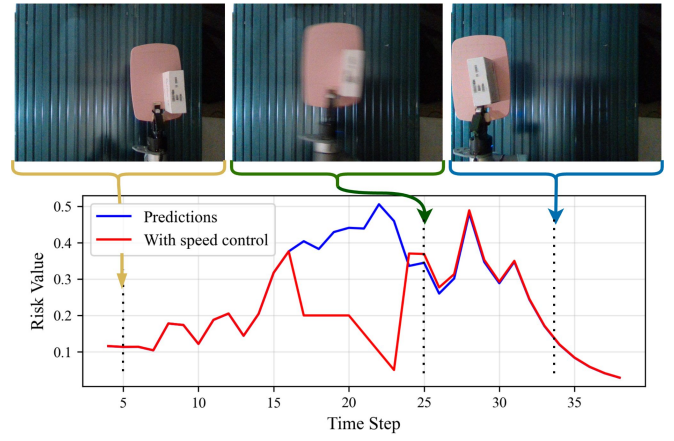


Fig. 6: Trajectory showing a real-world deployment scenario where joint speed is set based on the predicted risk value which in turn mitigates subsequent risk after time step 20.

determining which relative motions are safety-critical. Second, while our method predicts when intervention is needed, it does not address a general *recovery strategies*, which are application-specific and may require domain knowledge or specialized hardware. Third, our relative motion estimation relies on 2D ICP over projected contours, which may degrade under occlusion, poor lighting, or visual ambiguity; depth sensing with 3D ICP could provide more robustness at increased computational cost. Finally, our historical backtracking data collection is efficient in simulation but is labor-intensive in the real-world due to repeated rollouts and resets.

Can we model the system dynamics analytically and predict failure from initial conditions and planned trajectories? The contact dynamics between objects and carriers involve complex, nonlinear interactions governed by friction, surface geometry, and material properties that are not reliably identifiable online and are difficult to model accurately. This motivates a learning-based approach that infers failure risk directly from observed data.

Why use interpolated values for the risk curve? Our primary goal is to accurately identify the critical boundary t_{stop} . Intermediate risk values act as supervisory signals that help the model learn the temporal progression toward this boundary, approximating the continuous degradation of recoverability as destabilizing factors (momentum, contact instability) accumulate, and the safety margin diminishes smoothly. Smooth continuous labels provide better learning signals than discrete jumps. Although exact intermediate values are not physically measured, they provide a relative ordering of risk that helps the model distinguish early-stage stable conditions from late-stage critical conditions. The classification loss enforces learning of the binary boundary ($R_i = 0.5$ at t_{stop}), while the regression loss encourages temporal consistency. In preliminary experiments, spline interpolation outperformed step and exponential labeling in striking best balance in intervention timeliness and precision (see supplementary website for details).

Why use a fixed threshold? While failure patterns vary across rollouts (e.g., different object dynamics), our model learns to map diverse observable patterns to a calibrated risk

scale where 0.5 consistently represents the t_{stop} boundary. The network implicitly learns: “when I observe motion pattern X, how close am I to t_{stop} ?” A fixed threshold enables consistent interpretation: operators know that $R_i > 0.5$ means past the “point of no return,” simplifying deployment across object types and speeds without per-scenario tuning.

VI. CONCLUSION

In this paper, we investigate how proactive failure prediction can balance cycle time and risk management in non-prehensile lift-and-place material handling tasks. We propose PREFAIL, which extracts relative motion from images to capture scene dynamics and predict risk in advance. We demonstrate that PREFAIL achieves an effective guarantee of cycle time and manipulation reliability by adapting execution speed based on risk assessment. Our experimental investigation focused on object transport using a part carrier. However, we believe that the methodology presented in this paper will advance proactive robotic failure prediction for a wide variety of high-speed material handling applications.

ACKNOWLEDGMENT

We acknowledge support from the National Artificial Intelligence Research Resource (NAIRR) Pilot program. We thank Jeremy Morgan, Jason Chen, Siheng Zhao, Minjune Hwang, and Zhehui Huang for their helpful technical advice and discussions.

REFERENCES

- [1] M. T. Mason, “Mechanics and planning of manipulator pushing operations,” *The International Journal of Robotics Research*, 1986.
- [2] K. M. Lynch and M. T. Mason, “Stable pushing: Mechanics, controllability, and planning,” *The international journal of robotics research*, 1996.
- [3] F. Ruggiero, V. Lippiello, and B. Siciliano, “Nonprehensile dynamic manipulation: A survey,” *IEEE Robotics and Automation Letters (RA-L)*, 2018.
- [4] J. Duan, W. Pumacay, N. Kumar, Y. R. Wang, S. Tian, W. Yuan, R. Krishna, D. Fox, A. Mandlekar, and Y. Guo, “Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [5] S. Thoduka, N. Hochgeschwender, J. Gall, and P. G. Plöger, “A multimodal handover failure detection dataset and baselines,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [6] G. LeMasurier, A. Gautam, Z. Han, J. W. Crandall, and H. A. Yanco, “Reactive or proactive? how robots should explain failures,” in *International Conference on Human-Robot Interaction (HRI)*, 2024.
- [7] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. A. Ambrus, H. Nishimura, and M. Itkina, “Can We Detect Failures Without Failure Data? Uncertainty-Aware Runtime Failure Detection for Imitation Learning Policies,” in *Robotics: Science and Systems (RSS)*, 2025.
- [8] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, “What matters in learning from offline human demonstrations for robot manipulation,” in *Conference on Robot Learning (CoRL)*, 2022.
- [9] X. Jia, D. Blessing, X. Jiang, M. Reuss, A. Donat, R. Lioutikov, and G. Neumann, “Towards diverse behaviors: A benchmark for imitation learning with human demonstrations,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [10] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *International Journal of Robotics Research (IJRR)*, 2025.
- [11] C. Agia, R. Sinha, J. Yang, Z. Cao, R. Antonova, M. Pavone, and J. Bohg, “Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress,” in *Conference on Robot Learning (CoRL)*, 2025.
- [12] Z. Zheng, Q. Feng, H. Li, A. Knoll, and J. Feng, “Evaluating uncertainty-based failure detection for closed-loop llm planners,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [13] A. Inceoglu, E. E. Aksoy, and S. Sariel, “Multimodal detection and classification of robot manipulation failures,” *IEEE Robotics and Automation Letters (RA-L)*, 2024.
- [14] Y. Ma, J. Liu, I. Mamaev, and A. Morozov, “Multimodal failure prediction for vision-based manipulation tasks with camera faults,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [15] Z. Liu, A. Bahety, and S. Song, “Reflect: Summarizing robot experiences for failure explanation and correction,” in *Conference on Robot Learning (CoRL)*, 2023.
- [16] J. H. Kang, N. Dhanaraj, S. Wadaskar, and S. K. Gupta, “Using large language models to generate and apply contingency handling procedures in collaborative assembly applications,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [17] P. Pramanick and S. Rossi, “Multimodal coherent explanation generation of robot failures,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [18] Y. Dai, J. Lee, N. Fazeli, and J. Chai, “Racer: Rich language-guided failure recovery policies for imitation learning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [19] R. Sinha, A. Elhafsi, C. Agia, M. Foutter, E. Schmerling, and M. Pavone, “Real-Time Anomaly Detection and Reactive Planning with Large Language Models,” in *Robotics: Science and Systems (RSS)*, 2024.
- [20] N. Sogi, H. Oyama, T. Shibata, and M. Terao, “Future predictive success-or-failure classification for long-horizon robotic tasks,” in *International Joint Conference on Neural Networks (IJCNN)*, 2024.
- [21] E. Zhou, Q. Su, C. Chi, Z. Zhang, Z. Wang, T. Huang, L. Sheng, and H. Wang, “Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [22] P. W. Huang and K.-J. Chung, “Task failure prediction for wafer-handling robotic arms by using various machine learning algorithms,” *Measurement and Control*, 2021.
- [23] A. Parashar, K. Garg, J. Zhang, and C. Fan, “Failure prediction from limited hardware demonstrations,” in *2025 61st Allerton Conference on Communication, Control, and Computing Proceedings*, 2025.
- [24] J.-e. Lee, R. Sun, A. Byland, and L. Sentis, “Grasp failure constraints for fast and reliable pick-and-place using multi-suction-cup grippers,” *IEEE Transactions on Automation Science and Engineering*, 2025.
- [25] S. Sagar, J. Duan, S. Vasudevan, Y. Zhou, H. B. Amor, D. Fox, and R. Senanayake, “Uncovering robot vulnerabilities through semantic potential fields,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [26] H. Liu, Y. Zhang, V. Betala, E. Zhang, J. Liu, C. Ding, and Y. Zhu, “Multi-task interactive robot fleet learning with visual world models,” in *Conference on Robot Learning (CoRL)*, 2025.
- [27] N. He, S. Li, Z. Li, Y. Liu, and Y. He, “Rediffuser: reliable decision-making using a diffuser with confidence estimation,” in *International Conference on Machine Learning (ICML)*, 2024.
- [28] L. Yang, Z. Zhang, Z. Zhang, X. Liu, M. Xu, W. Zhang, C. Meng, S. Ermon, and B. Cui, “Consistency flow matching: Defining straight flows with velocity consistency,” *arXiv preprint arXiv:2407.02398*, 2024.
- [29] B. Charpentier, O. Borchert, D. Zügner, S. Geisler, and S. Günnemann, “Natural posterior network: Deep bayesian predictive uncertainty for exponential family distributions,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [30] R. Römer, A. Kobras, L. Worbis, and A. P. Schoellig, “Failure prediction at runtime for generative robot policies,” in *Neural Information Processing Systems (NeurIPS)*, 2025.
- [31] M. Miller, B. Schölkopf, and S. Guo, “Counterfactual reasoning: an analysis of in-context emergence,” in *Neural Information Processing Systems (NeurIPS)*, 2025.
- [32] K. Erdogan, “Spline interpolation techniques,” *Journal of Technical Science and Technologies*, 2013.
- [33] P. J. Besl and N. D. McKay, “Method for registration of 3-d shapes,” in *Sensor fusion IV: control paradigms and data structures*, 1992.
- [34] J. Yang, H. Li, and Y. Jia, “Go-icp: Solving 3d registration efficiently and globally optimally,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2013.
- [35] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollar, and C. Feichtenhofer, “Sam 2: Segment anything in images and videos,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [36] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [37] A. M. Lamb, A. G. Alias Parth Goyal, Y. Zhang, S. Zhang, A. C. Courville, and Y. Bengio, “Professor forcing: A new algorithm for training recurrent networks,” *Neural Information Processing Systems (NeurIPS)*, 2016.
- [38] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [39] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.
- [40] N. Sarawgi, O. M. Manyar, F. Wang, T. H. Nguyen, D. Seita, and S. K. Gupta, “Expanding Picking Actions for Time-Efficient Online 3D Bin Packing,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [41] T. G. W. Lum, A. H. Li, P. Culbertson, K. Srinivasan, A. Ames, M. Schwager, and J. Bohg, “Get a grip: Multi-finger grasp evaluation at scale enables robust sim-to-real transfer,” in *Conference on Robot Learning (CoRL)*, 2025.